

MNC Livestock Report

Technical Guide

Mobile boma movements, cattle counts, livestock predation, and illegal grazing reporting

Generated August 08, 2026

Workflow id: **livestock_monitoring**

1. Overview

The **livestock_monitoring** workflow (repository: `mnc_livestock_report`) fetches livestock-related events from EarthRanger for a specified time window — specifically **mobile_boma_rep**, **cattle_count**, **livestock_predation_rep**, and **illegal_grazing_rep** event types — and routes them into four independent reporting branches. In parallel, the workflow downloads MNC conservancy boundary and parcels geospatial files from Dropbox and builds a shared set of map base layers (conservancy boundary, colour-coded grazing zones, parcels) used by all three maps. Three of the four branches, plus all three maps, are additionally wrapped as widgets on the workflow's dashboard.

The workflow delivers:

- **mobile_boma_movement_summary_table.csv** — daily count of mobile boma movement events
- **total_cattle_count_summary_table.csv** — cattle counts per grazing zone per date, with a workflow-computed total column
- **total_livestock_predation_summary_table.csv** — daily count of livestock predation events
- **livestock_predation_summary_table.csv** — predation incidents by species, suspected predator, and number of animals affected
- **boma_movement_map.html / .png** — point map of mobile boma locations over the shared base layers
- **livestock_predation_events.html / .png** — point map of predation incidents over the shared base layers, coloured by species
- **illegal_grazing_map.html / .png** — point map of illegal grazing incidents over the shared base layers
- **A dashboard** — five widgets: the three maps above, plus the cattle count and livestock predation count-summary tables

Output summary

Output	Type	Source event type	Dashboard widget?
mobile_boma_movement_summary_table.csv	CSV	mobile_boma_rep	No
total_cattle_count_summary_table.csv	CSV	cattle_count	Yes (table)
total_livestock_predation_summary_table.csv	CSV	livestock_predation_rep	Yes (table)
livestock_predation_summary_table.csv	CSV	livestock_predation_rep	No
boma_movement_map.html / .png	Map	mobile_boma_rep	Yes (map)
livestock_predation_events.html / .png	Map	livestock_predation_rep	Yes (map)
illegal_grazing_map.html / .png	Map	illegal_grazing_rep	Yes (map)

Note: `total_livestock_predation_summary_table.csv` (daily counts) and `livestock_predation_summary_table.csv` (detailed per-event records) are two distinct outputs from the same branch — only the former has a dashboard widget, titled “Livestock Predation Summary”.

2. Dependencies

2.1 Python packages

Package	Version	Channel
ecoscope-platform	>=2.15.0, <2.16.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge

Note: **ecoscope-platform** replaces the previously separate **ecoscope-workflows-core** and **ecoscope-workflows-ext-ecoscope** packages. The **ecoscope-workflows-ext-mep** and **ecoscope-workflows-ext-big-life** packages, previously listed, are no longer required. Several tasks that used to resolve from the default task namespace are now referenced by their fully qualified module path (e.g. `ecoscope_workflows_ext_ste.tasks.io.fetch_and_persist_file`, `ecoscope_workflows_ext_custom.tasks.results.draw_map`) — this is a housekeeping change with no behavioural effect.

2.2 Connections and external assets

Asset	Task / Source	Purpose
EarthRanger	set_er_connection	Fetch event records and resolve event detail display titles (used in all <code>process_events_details</code> calls)
mnc_conservancy.gpkg	fetch_and_persist_file (Dropbox)	MNC community conservancy boundary, used to derive the conservancy outline, grazing zone, and Mara North Conservancy layers.
mnc_across_the_river_parcel.gpkg	fetch_and_persist_file (Dropbox)	MNC across-the-river land parcels. Used as an additional polygon layer on all three maps.

Note: Both Dropbox files are downloaded with `overwrite_existing: false` and `retries: 3`. If the files already exist in `ECOSCOPE_WORKFLOWS_RESULTS` from a previous run, the download is skipped.

2.3 Grouper

The workflow uses an **empty grouper list** (groupers: []). All event records are processed as a single undivided dataset — no fan-out or per-group branching is applied to the data. The grouper is passed through to the temporal index and the dashboard only.

3. Geospatial Asset Pipeline

Before any event data is fetched, the workflow downloads and prepares all geospatial base layers. These layers are shared across all three maps produced by the workflow. This pipeline was substantially reworked from the previous revision — see the notes at the end of this section.

3.1 Conservancy boundary and grazing zone filters

Step	Task	Detail
1	fetch_and_persist_file	Download mnc_conservancy.gpkg from Dropbox to ECOSCOPE_WORKFLOWS_RESULTS (overwrite_existing: false, retries: 3).
2	load_df	Load the gpkg into a GeoDataFrame (layer: null, deserialize_json: false).
3	fix_invalid_geometries	Repair any invalid geometries in the loaded boundary GeoDataFrame.
4	filter_df	Filter to grazing_zone == 'Conservancy' → filter_conservancy_boundary, used for the conservancy outline layer.
5	filter_df	Filter to name == 'Mara North Conservancy' → filter_mara_north, used to compute the shared map zoom/extent (Section 4.4).
6	filter_df	Filter to grazing_zone != 'Conservancy' → filter_grazing_zones, the coloured grazing-zone polygons.

3.2 Grazing zone colouring and legend

Step	Task	Detail
1	apply_color_map	Colour filter_grazing_zones by the grazing_zone column using the GnBu colormap → output column zone_color .
2	build_legend_values_from_column	Build the map legend entries directly from the grazing_zone / zone_color columns (sort: ascending).

Note: Grazing zone colours and legend entries are now generated dynamically from whatever zone names are present in the data, instead of a hardcoded style dictionary keyed on six fixed zone names. If new zones are added or renamed in the source boundary file, the map colouring and legend adapt automatically.

3.3 Parcels layer

Step	Task	Detail
1	fetch_and_persist_file	Download mnc_across_the_river_parcel.gpkg from Dropbox (overwrite_existing: false, retries: 3).
2	load_df	Load the parcels gpkg into a GeoDataFrame.

3.4 Map layers

All three static base layers are now created with the same task, **create_geojson_layer**:

Layer	Style	Legend
create_conservancy_layer	Unfilled, stroked grey outline (169,169,169,255), width 1.75	“Boundaries” — Conservancy Boundary (#a9a9a9)
create_grazing_zones_layer	Filled and stroked, opacity 0.5, fill/line colour from zone_color	“Grazing Zones” — dynamic, from Section 3.2
create_parcels_layer	Filled (189,183,107,60) / stroked (189,183,107,255), width 1.5	“Boundaries” — Parcels (#bdb76b)

Note: The previous revision built these same three concepts very differently: it split the boundary file into a dict of six hardcoded zones (split_gdf_by_column / annotate_gdf_dict_with_geom_type / create_deckgl_layers_from_gdf_dict) and rendered a separate text-label layer showing the conservancy name at its centroid (create_custom_text_layer). Both the zone dict-splitting approach and the conservancy name text-label layer have been removed in this revision — maps no longer show a name label on the conservancy polygon. The parcels layer no longer needs a separate geometry-type detection step (get_gdf_geom_type) before rendering.

4. Event Ingestion Pipeline

All four reporting branches share a common event ingestion and normalisation pipeline.

4.1 Event retrieval

Parameter	Value
Task	get_events
event_types	mobile_boma_rep, cattle_count, livestock_predation_rep, illegal_grazing_rep
Columns retained	id, time, event_type, event_category, reported_by, serial_number, geometry, created_at, event_details, patrols
include_details	true
raise_on_empty	true
include_null_geometry	false
include_updates	false
include_related_events	false
include_display_values	false
force_point_geometry	true

Note: force_point_geometry: true normalises all event geometries to points before they reach the branch pipelines and the map layers below.

4.2 Date extraction and temporal indexing

Step	Task	Detail
1	extract_column_as_type	Extract the time column as output_type: date into a new column named date .
2	add_temporal_index	Add a temporal index using time_col: date , groupers: [], cast_to_datetime: true, format: mixed. Produces the shared events_temporal DataFrame.

4.3 Common branch normalisation pattern

Every branch applies the same four-step normalisation after filtering:

Step	Task	Detail
1	filter_df	Filter events_temporal by event_type (op: equal, reset_index: true).
2	process_events_details	Resolve event detail field IDs to display titles (map_to_titles: true, ordered: true). Requires the ER client.
3	normalize_json_column	Flatten the event_details JSON column (skip_if_not_exists: true, sort_columns: true).

Step	Task	Detail
4	drop_column_prefix	Remove the event_details__ prefix from all flattened columns (duplicate_strategy: keep_original).

Note: Because map_to_titles is true, the flattened column names are the human-readable field titles from EarthRanger (e.g. 'Mobile Boma Zone', 'Livestock Species'). All downstream map_columns steps reference these titles directly.

4.4 Shared map zoom and extent

A task-group titled “**Map Zoom & Extent**” computes a single view state, shared by all three maps in this workflow:

Step	Task	Detail
1	envelope_gdf	Compute the bounding envelope of filter_mara_north (the Mara North Conservancy boundary, Section 3.1).
2	compute_view_state_from_gdf	Derive a centre point and zoom level from that envelope (pitch: 0, bearing: 0, max_zoom: 15).

Note: This replaces the previous revision's approach, which computed a zoom value from the union of grazing zone geometries (view_state_deck_gdf on overall_grazing_zones) and used it only for the boma movement and illegal grazing maps — the livestock predation map used a separate, hand-picked fixed coordinate and zoom (lon: 35.2093, lat: -1.2578, zoom: 9.75). All three maps now share the same conservancy-derived view state.

5. Branch 1 — Mobile Boma Movements

Filters **mobile_boma_rep** events and produces a daily movement summary table and a point map of boma locations.

5.1 Normalisation

Steps 1–4 follow the common normalisation pattern in Section 4.3.

5.2 Column selection

Task: **map_columns** (raise_if_not_found: false, rename_columns: {}). The following columns are retained:

Column retained	Notes
id	Used for counting events in the summary
date	Used for grouping in the summary table
event_type	Retained but not used for map colouring
geometry	Used for the map
Date of Relocation	Event detail field (title)
Electric Boma Status	Event detail field (title)
Mobile Boma Zone	Event detail field (title)
Nature of the Site	Event detail field (title)
Reason for relocation	Event detail field (title)

5.3 Summary table

Step	Task	Detail
1	summarize_df	Group by date ; compute nunique(id) displayed as boma_events (decimal_places: 0). reset_index: true.
2	persist_df	Save as mobile_boma_movement_summary_table.csv .

Note: The previous revision appended a grand “Total” row via `add_totals_row` before persisting. That step has been removed — the CSV now contains only one row per date.

5.4 Map

Step	Task	Detail
1	create_scatterplot_layer	Render points from the <code>map_mobile_boma</code> table: fixed navy fill/line colour (0,0,128), <code>get_radius</code> : 3, <code>opacity</code> : 0.55, <code>stroked</code> : true. Legend title: “Movements”, single entry “Boma movement”.
2	combine_deckgl_map_layers	Static layers: <code>create_grazing_zones_layer</code> , <code>create_parcels_layer</code> , <code>create_conservancy_layer</code> (Section 3.4). Grouped: the boma point layer.

Step	Task	Detail
3	draw_map	Render map (max_zoom: 10, legend placement: bottom-right, tile_layers from configure_base_maps, view_state from the shared Map Zoom & Extent group, Section 4.4).
4	persist_text	Save HTML to boma_movement_map.html .
5	create_map_widget_single_view	Wrap the map as a dashboard widget titled "Mobile Boma Movement Map".
6	html_to_png	Convert to PNG (device_scale_factor: 2.0, wait: 40 s, max_concurrent_pages: 1).

6. Branch 2 — Cattle Counts

Filters **cattle_count** events and produces a table of cattle counts broken down by grazing zone per date, with a workflow-computed total. This branch produces no map.

6.1 Normalisation

Steps 1–4 follow the common normalisation pattern in Section 4.3.

6.2 Column selection and renaming

Task: **map_columns** (raise_if_not_found: false). The source **total_cattle_counted_from_all_zones** column is now dropped rather than retained — the workflow computes its own total instead (Section 6.3).

Source column (title after prefix drop)	Renamed to
date	date (retained, not renamed)
# cattle in Zone 1 mobile boma	zone_1
# cattle in Zone 2/3 mobile boma	zone_2_3
# cattle in Zone 4	zone_4

6.3 Total, display renaming, and persistence

Step	Task	Detail
1	convert_column_values_to_numeric	Coerce zone_1, zone_2_3, and zone_4 to numeric.
2	ecoscope_workflows_ext_mnc.tasks.aggregation.apply_arithmetic_operation_over_rows	Sum zone_1 + zone_2_3 + zone_4 row-wise into a new total column (operation: add).
3	map_columns	Rename columns to display-friendly headers: date→Date, zone_1→Zone 1, zone_2_3→Zone 2/3, zone_4→Zone 4, total→Total.
4	persist_df	Save as total_cattle_count_summary_table.csv , using the display-renamed table.
5	draw_table	Render the display-renamed table as an HTML widget (widget_id: "Total Cattle Count Summary"; sorting and filtering enabled; download disabled).
6	persist_text	Save the rendered HTML as a text file (filename: total_cattle_count_summary_table.html).
7	create_table_widget_single_view	Wrap the persisted HTML into a dashboard widget titled "Total Cattle Count Summary".

Note: Previously, the source event's own total_cattle_counted_from_all_zones field was trusted and simply renamed to total_count. The workflow now computes the total itself from the three zone counts, and the table gains a dashboard widget it did not have before.

7. Branch 3 — Livestock Predation

Filters **livestock_predation_rep** events and produces a point map, a daily count-summary table, and a detailed record-level table.

7.1 Normalisation

Steps 1–4 follow the common normalisation pattern in Section 4.3.

7.2 Column selection

Task: **map_columns** (raise_if_not_found: false, rename_columns: {}). The following columns are retained and shared by the map, the count-summary table, and the detailed table:

Column retained	Notes
id	Row identifier
date	Event date
event_type	Retained but not used for map colouring
geometry	Used for the map
Livestock Species	Event detail field (title)
Suspected Predator	Event detail field (title)
Total livestock affected	Event detail field (title)

7.3 Map

Step	Task	Detail
1	format_text_column	Capitalise the Livestock Species column (method: capitalize), so inconsistent casing in the source data doesn't split a species into multiple colours/legend entries.
2	apply_color_map	Colour points by the capitalised Livestock Species using the Set3 colormap → output column colors .
3	create_scatterplot_layer	Render points: fill/line colour from colors, get_radius: 3, opacity: 0.55, stroked: true. Legend title: "Livestock Species", entries generated from the species/colour columns (sort: ascending).
4	combine_deckgl_map_layers	Static layers: create_grazing_zones_layer, create_parcel_layer, create_conservation_layer (Section 3.4). Grouped: the predation point layer.
5	draw_map	Render map (max_zoom: 10, legend: bottom-right, view_state from the shared Map Zoom & Extent group, Section 4.4).
6	persist_text	Save HTML to livestock_predation_events.html .

Step	Task	Detail
7	create_map_widget_single_view	Wrap the map as a dashboard widget titled “Livestock Predation Events Map”.
8	html_to_png	Convert to PNG (device_scale_factor: 2.0, wait: 40 s).

Note: Two things changed from the previous revision: (1) this map now uses the same shared conservancy-derived view state as the other two maps, instead of a fixed coordinate/zoom; and (2) it now sits on the full three-layer base (grazing zones + parcels + conservancy) rather than the conservancy-boundary-only layer set — grazing zone colours are now visible under the predation points.

7.4 Summary tables

The livestock predation branch produces two independent CSVs from the same normalised DataFrame.

7.4a total_livestock_predation_summary_table.csv (dashboard widget: “Livestock Predation Summary”)

Step	Task	Detail
1	summarize_df	Group by date ; compute nunique(id) displayed as livestock_predation_events (decimal_places: 0). reset_index: true.
2	map_columns	Rename columns to display-friendly headers: date→Date, livestock_predation_events→Livestock Predation Events.
3	persist_df	Save as total_livestock_predation_summary_table.csv , using the display-renamed table.
4	draw_table	Render the display-renamed table as an HTML widget (widget_id: “Livestock Predation Summary”).
5	persist_text	Save the rendered HTML (filename: livestock_predation_summary_table.html).
6	create_table_widget_single_view	Wrap the persisted HTML into a dashboard widget titled “Livestock Predation Summary”.

Note: The previous revision appended a grand “Total” row (add_totals_row) here instead of renaming columns and creating a widget. The output filename is unchanged, but its content and presentation differ.

Note: This step's persisted HTML filename (livestock_predation_summary_table.html) differs only by extension from the unrelated detail CSV in Section 7.4b (livestock_predation_summary_table.csv) — don't confuse the two when browsing ECOSCOPE_WORKFLOWS_RESULTS.

7.4b livestock_predation_summary_table.csv (no dashboard widget)

Step	Task	Detail
1	map_columns	Retain date, Livestock Species, Suspected Predator, Total livestock affected. Rename: Livestock Species → livestock_species, Suspected Predator → suspected_predator, Total livestock affected → total_livestock_affected. (raise_if_not_found: false)
2	fill_missing_values	Replace missing values in suspected_predator and livestock_species with 'Unknown' (subset: both columns).
3	replace_column_values	Map 'Other (specify in comments)' → 'Unknown' in the suspected_predator column (inplace: true, errors: ignore).
4	ecoscope_workflows_ext_mnc.tasks.transformations.convert_columns_to_int	Cast total_livestock_affected to integer (errors: coerce, fill_value: 0).
5	persist_df	Save as livestock_predation_summary_table.csv .

Note: Unlike the other three summary tables in this workflow, this detail table is not passed through a display-renaming step — its columns remain snake_case (livestock_species, suspected_predator, total_livestock_affected), and it has no dashboard widget.

8. Branch 4 — Illegal Grazing

Filters `illegal_grazing_rep` events and produces a point map of illegal grazing incidents. This branch produces no CSV summary table.

8.1 Normalisation

Steps 1–4 follow the common normalisation pattern in Section 4.3.

8.2 Column selection

Task: `map_columns` (`raise_if_not_found`: `false`, `rename_columns`: `{}`). The following columns are retained:

Column retained	Notes
<code>date</code>	Event date
<code>event_type</code>	Retained but not used for map colouring
<code>geometry</code>	Used for the map
<code>Herd Zone</code>	Event detail field (title)
<code>Landowner name</code>	Event detail field (title)
<code>action taken</code>	Event detail field (title)

8.3 Map

Step	Task	Detail
1	<code>create_scatterplot_layer</code>	Render points: fixed navy fill/line colour (0,0,128), <code>get_radius</code> : 3, <code>opacity</code> : 0.55, <code>stroked</code> : <code>true</code> . Legend title: “Activity”, single entry “Illegal grazing”.
2	<code>combine_deckgl_map_layers</code>	Static layers: <code>create_grazing_zones_layer</code> , <code>create_parcels_layer</code> , <code>create_conservancy_layer</code> (Section 3.4). Grouped: the illegal grazing point layer.
3	<code>draw_map</code>	Render map (<code>max_zoom</code> : 10, <code>legend</code> : bottom-right, <code>view_state</code> from the shared Map Zoom & Extent group, Section 4.4).
4	<code>persist_text</code>	Save HTML to <code>illegal_grazing_map.html</code> .
5	<code>create_map_widget_single_view</code>	Wrap the map as a dashboard widget titled “Illegal Grazing Events Map”.
6	<code>html_to_png</code>	Convert to PNG (<code>device_scale_factor</code> : 2.0, <code>wait</code> : 40 s).

Note: The legend title changed from “Illegal grazing” to “Activity” (the single legend entry is still labelled “Illegal grazing”). More significantly, this map’s static base layers now include the parcels layer — previously parcels were deliberately excluded from this map; all three maps now share the identical three-layer base.

9. Output Files

All outputs are written to **ECOSCOPE_WORKFLOWS_RESULTS**.

9.1 CSV tables

File	Branch	Columns	Description
mobile_boma_movement_summary_table.csv	Mobile Boma	date, boma_events	Daily boma event counts
total_cattle_count_summary_table.csv	Cattle Count	Date, Zone 1, Zone 2/3, Zone 4, Total	Cattle counts per zone per date, with a workflow-computed total
total_livestock_predation_summary_table.csv	Livestock Predation	Date, Livestock Predation Events	Daily unique predation event count
livestock_predation_summary_table.csv	Livestock Predation	date, livestock_species, suspected_predator, total_livestock_affected	Predation records by species and predator

9.2 Maps

File	Branch	Coloured by	Base layers
boma_movement_map.html / .png	Mobile Boma	fixed (navy)	Grazing zones, parcels, conservancy boundary
livestock_predation_events.html / .png	Livestock Predation	Livestock Species	Grazing zones, parcels, conservancy boundary
illegal_grazing_map.html / .png	Illegal Grazing	fixed (navy)	Grazing zones, parcels, conservancy boundary

9.3 Dashboard widget HTML

The two count-summary tables (cattle count, livestock predation) each have a matching rendered HTML table persisted (via `draw_table` → `persist_text`) alongside their CSV. These HTML files are the data source referenced by the corresponding `create_table_widget_single_view` widget — they are not intended to be opened directly.

10. Workflow Execution Logic

10.1 Global skip conditions

This workflow now defines a single **task-instance-defaults** block at the top of the spec, which applies the same skipif conditions to every task automatically. Individual tasks no longer repeat their own skipif block:

Condition	Behaviour
any_is_empty_df	Skip this task if any input DataFrame is empty
any_dependency_skipped	Skip this task if any upstream dependency was skipped

Note: This is a behaviour-preserving simplification over the previous spec, which declared the identical skipif block on every task individually. Because the conditions are unchanged, each of the four branches still propagates skips independently — for example, if no `cattle_count` events are returned, only the cattle count branch (and its widget) is skipped; the other three branches continue normally.

10.2 Four independent branches

After the shared ingestion pipeline produces **events_temporal**, the workflow splits into four fully independent branches. Each branch reads directly from `events_temporal` with no cross-branch dependencies:

Branch	Filter value	CSV output	Map / table widgets
Mobile Boma	<code>mobile_boma_rep</code>	<code>mobile_boma_movement_summary_table.csv</code>	Map widget only
Cattle Count	<code>cattle_count</code>	<code>total_cattle_count_summary_table.csv</code>	Table widget
Livestock Predation	<code>livestock_predation_rep</code>	<code>total_livestock_predation_summary_table.csv</code> + <code>livestock_predation_summary_table.csv</code>	Map + table widget (count table only)
Illegal Grazing	<code>illegal_grazing_rep</code>	—	Map widget only

10.3 No mapvalues or fan-out

This workflow processes all records as a single batch. There is no **mapvalues**, **split_groups**, or **zip_groupbykey** — every task runs exactly once.

10.4 HTML to PNG conversion

Three maps are rendered as HTML and then converted to PNG using **html_to_png**. All three use the same conversion settings: `device_scale_factor: 2.0`, `wait_for_timeout: 40 000 ms`, `max_concurrent_pages: 1`, `full_page: false`. This happens in addition to, not instead of, each map's dashboard widget.

10.5 Dashboard

The workflow concludes with **gather_dashboard** (id: `mnc_events_dashboard`, name: "MNC event report dashboard"), which packages workflow details, time range, groupers, and the **widgets** list. The widgets list now references five

widgets — the three map widgets and the two table widgets (cattle count, livestock predation count-summary) — where previously this list was empty.

11. Software Versions

Package	Version pinned in spec.yml
ecoscope-platform	>=2.15.0, <2.16.0
ecoscope-workflows-ext-custom	0.1.0rc14.*
ecoscope-workflows-ext-ste	0.0.0rc1.*
ecoscope-workflows-ext-mnc	1.0.0.*
pydeck	0.9.2
opentelemetry-sdk	>=1.20.0, <2.0.0